Università di Catania

Dipartimento di
Ingegneria Elettrica Elettronica e Informatica

Advanced Programming Languages

C++: data types

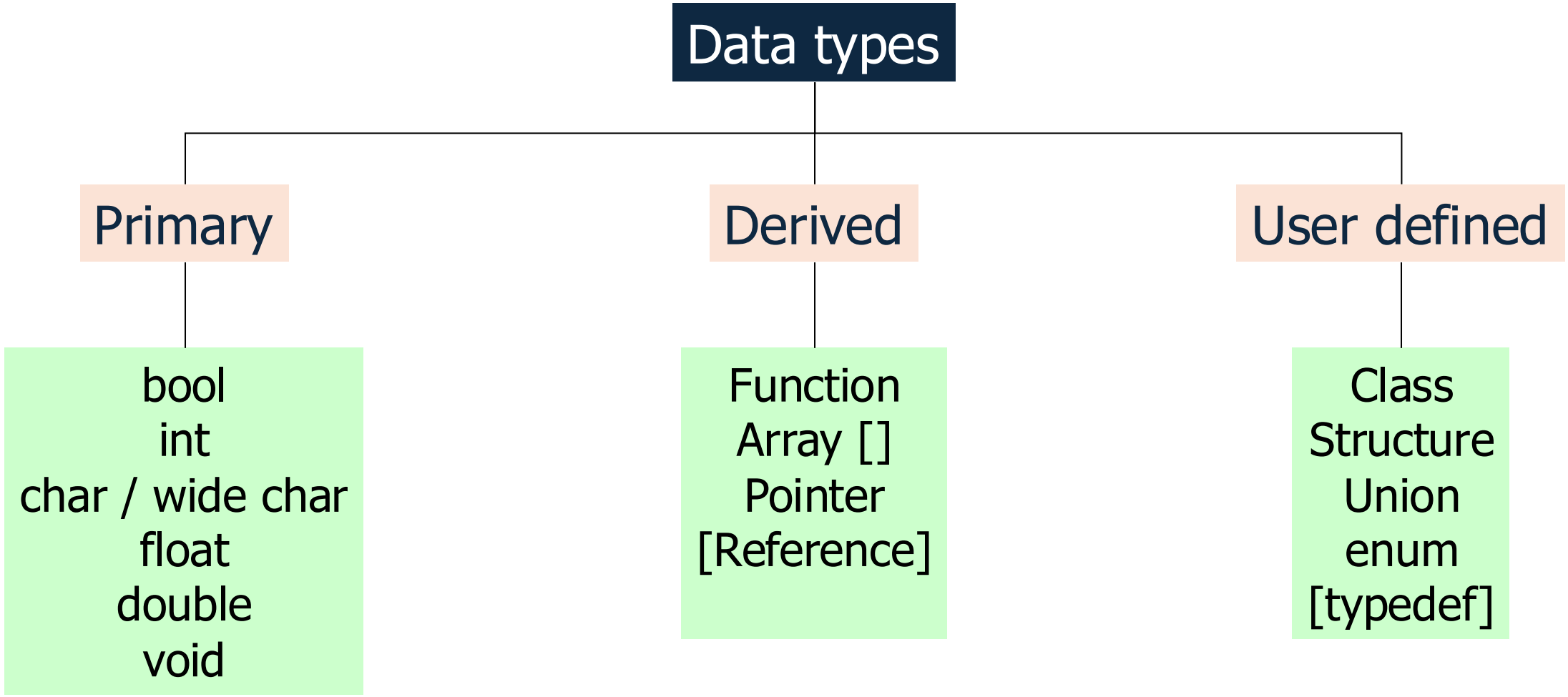
Docente:
Marco Grassia
marco.grassia@unict.it

A.A. 2025/2026

C++ type system

- The type system is **strongly typed & statically typed**
  - Every object has a **fixed type** that **never changes** after declaration
  - The compiler (**statically**) **enforces strict rules about type usage**. Operations must be valid for the operand types, and **implicit conversions are limited**
- **In C++, there is no universal base type** (unlike Java or C#)
- **Two main type categories**:
  - **Scalar types**: hold a single value
  - **Compound types**: not scalar

Data types in C++



Fundamental types

Data types

http://www.cplusplus.com/doc/tutorial/variables/

Built-in (aka fundamental) Types

- "**primitive**" or "intrinsic" **means** these types are **not objects**
- Here are (some of) the most used ones
- Note that the **exact bit ranges** here **are platform and compiler dependent**!

Name	Name	Value
char	unsigned char	8-bit integer
short	unsigned short	16-bit integer
int	unsigned int	32-bit integer
long	unsigned long	64-bit integer
bool	-	true or false

Name	Value
float	32-bit floating point
double	64-bit floating point
long long	128-bit integer
long double	128-bit floating point

Primitive types

- Examples of declaration of variables with different types

- Syntax:

  <type> <identifier>;

  <type> <identifier> = <initial value>;

```
// x is a floating point variable
float x;

// y is an integer variable initialized to 7
int y = 7;

// c is a character variable
char c;

// z is a double precision floating point variable
double z;

// void is a 'no-value' type
void sub();
// sub is a function (procedure) that takes
// no arguments and returns no value
```

Integer types

32 bit systems:

- LP32: Win16 API
- ILP32: Win32 API & Unix and Unix-like systems (Linux, Mac OS X)

64 bit systems:

- LLP64: Win64 API
- LP64: Unix and Unix-like systems (Linux, Mac OS X)

Type specifier	Equivalent type	Width in bits by data model				
		C++ standard	LP32	ILP32	LLP64	LP64
signed char	signed char	at least **8**	**8**	**8**	**8**	**8**
unsigned char	unsigned char					
short	short int	at least **16**	**16**	**16**	**16**	**16**
short int						
signed short						
signed short int						
unsigned short	unsigned short int					
unsigned short int						
int	int	at least **16**	**16**	**32**	**32**	**32**
signed						
signed int						
unsigned	unsigned int					
unsigned int						

Integer types

32 bit systems:

- LP32: Win16 API
- ILP32: Win32 API & Unix and Unix-like systems (Linux, Mac OS X)

64 bit systems:

- LLP64: Win64 API
- LP64: Unix and Unix-like systems (Linux, Mac OS X)

Type specifier	Equivalent type	Width in bits by data model				
		C++ standard	LP32	ILP32	LLP64	LP64
long	long int	at least **32**	**32**	**32**	**32**	**64**
long int						
signed long						
signed long int						
unsigned long	unsigned long int					
unsigned long int						
long long	long long int (C++11)	at least **64**	**64**	**64**	**64**	**64**
long long int						
signed long long						
signed long long int						
unsigned long long	unsigned long long int (C++11)					
unsigned long long int						

C++ standard

- The exact bit ranges here are **platform and compiler dependent**
- **The exact byte count can be obtained the** sizeof(Type) operator

```
sizeof(Type);   // sizeof is an operator that returns the size
                // of a type or variable in bytes
```

- Besides the minimal bit counts, the C++ Standard guarantees that

$$1 == \text{sizeof}(\text{char}) \leq \text{sizeof}(\text{short}) \leq \text{sizeof}(\text{int}) \leq \text{sizeof}(\text{long}) \leq \text{sizeof}(\text{long long})$$

Need to be sure of integer sizes?

- Since C++, there are **type definitions that exactly specify exactly the bits used**
- These can be useful if you are planning to port code across CPU architectures (ex. Intel 64-bit CPUs to a 32-bit ARM on an embedded board) or when doing particular types of integer math

```
#include <cstdint>
```

Name	Name	Value
int8_t	uint8_t	8-bit integer
int16_t	uint16_t	16-bit integer
int32_t	uint32_t	32-bit integer
int64_t	uint64_t	64-bit integer

For a full list and description see:
http://www.cplusplus.com/reference/cstdint/

Bool examples

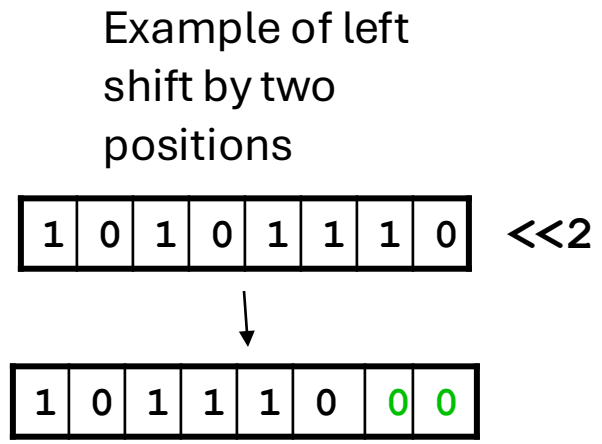
Represents **Boolean values**: true or false

Size: typically 1 byte

```
// Uninited bool vars
bool x1;      // x1 is an uninitialized bool variable
bool y1;      // y1 is an uninitialized bool variable

// Initialized bool vars
bool x = true;
bool y = false;

// Bool from comparisons
// Boolean equations return a bool
int a = 1;
int b = 1;
bool k = a==b;    // a è uguale a b?

bool q = (a!=0) && (b>7);
```

Bitwise Operators in C++

- Operate on the **binary representation of integers** (integral types only: int, unsigned, char, etc.), they **do not work on floating-point types**
- AND (&)
- OR (|)
- XOR (^)
- NOT (~)
- **Shift Left (<<)**: shifts bits left, filling with zeros
- **Shift Right (>>)**: shifts bits right. For signed types: **implementation-defined** (usually arithmetic shift)



Bitwise Operators in C++

```
unsigned int b = 0b11010010;
unsigned int c = 0b10101100;
unsigned int A;

A = b & c;  // bitwise and (AND)
A = b | c;  // bitwise or (OR)
A = b ^ c;  // bitwise xor (XOR)
A = ~b;     // bitwise not
A = b<<3;   // Shift left by 3 positions
A = b>>4;   // Shift right by 4 positions

// Esempi
(b>>2)&1;   // Value of the second bit
b | (1<<4); // Forces to one the fourth bit
b & ~(1<<4);    //  Forces to zero the fourth bit
(b>>8) & 0xFF;  // Extracts the higher bytes of the word
```

char and character types

- *char* **stores a single character (usually 1 byte)**
- Variants:
  - *signed char, unsigned char* (often used to represent bytes which does not belong to the language)
  - *wchar_t* (wide characters, implementation defined)
  - *char16_t, char32_t* (Unicode UTF-16 and UTF-32, C++11+)

```
// c is a character variable
char c = 'a';
std::printf("%c\n", c);
// c is converted to its ASCII integer code
int ci = int(c);
std::printf("%d\n", ci);

// k is a char var inited to ASCII code 32 (space)
char k = 32;
// ic is an int var inited to ASCII code of '$'
int ic = int('$');
// is ic an uppercase letter?
bool b = isupper(ic);
// is k a digit (0-9)?
bool d = isdigit(k);
std::println("Is uppercase: {}", b);
std::println("Is digit: {}", d);
```

Floating point types

- They represent **real numbers with different precision**
- The C++ standard **does not require adherence to the IEEE754 standard, but most compiler implementations adhere to it anyway**
  - `float`: single precision (~7 decimal digits), usually IEEE-754 32bit float
  - `double`: double precision (~15 decimal digits), usually IEEE-754 64bit
  - `long double`: extended precision, often a 80-bit x87 floating point type on x86 and x86-64 architectures, or same as `double`
- Support special values: $+\infty$, $-\infty$, *NaN* (Not a Number)
- Comparing floats with == is unsafe; use tolerance

Floating point types

+ - * / (and unary +/-), compound assignments += -= *= /=

increments/decrements ++ -- also work on floating types: ++x/x++ (add/subtract **1.0**), but prefer x += 1.0 for clarity in numeric code

^ is *not* exponentiation but **bitwise XOR** (for integers only), use **std::pow**

<cmath> provides the main math toolbox: pow, sqrt, cbrt, hypot, exp/log, sin/cos/tan, fma, etc

```
float a = 3.14e-2;
double b1 = 2.5e6;
double c1 = 2.5E-6;
std::println("a = {}, b1 = {}, c1 = {}", a, b1, c1);
std::println("b1 == c1? {}", b1 == c1);
```

```
a = 0.0314, b1 = 2500000, c1 = 2.51e-18
```

```
// ERROR: ^ is bitwise XOR, not exponentiation
// c1 = a^2 + b1^2;
c1 = pow(a, 2) + pow(b1, 2);
std::println("c1 = {}", c1);
```

```
c1 = 6250000000000.001
```

```
std::println("Almost equal? {}", (b1 - c1) < 1e-14);
```

```
Almost equal? false
```

Floating point types

<cmath> also has std::numeric_limits<T>::*epsilon*(), min(), max() for precision and range introspection

Be explicit with mixed types: float + double promotes to double (usual arithmetic conversions)

```
// Correct way to perform float comparisong using limits:
// epsilon is the difference between 1.0 and the next value
// representable by the floating-point type
// i.e., it is the smallest difference that is distinguishable from 0
    if (std::abs(a - b) < std::numeric_limits<float>::epsilon()) {
    std::println("a is approximately b");
}
```

Floating point types

<cmath> also has:

- round: rounds to the nearest integer, rounding halfway cases away from zero
- floor: rounds down to the nearest integer
- ceil: rounds up to the nearest integer
- trunc: rounds towards zero, removing the fractional part

```
std::println("round(2.5) = {}", std::round(2.5));
std::println("round(2.4) = {}", std::round(2.4));

std::println("floor(2.5) = {}", std::floor(2.5));

std::println("ceil(2.5) = {}", std::ceil(2.5));

std::println("trunc(2.5) = {}", std::trunc(2.5));
std::println("trunc(-2.5) = {}", std::trunc(-2.5));
```

```
round(2.5) = 3
round(2.4) = 2
floor(2.5) = 2
ceil(2.5) = 3
trunc(2.5) = 2
trunc(-2.5) = -2
```

Floating point types

<cmath> also has std::isfinite, std::isinf, std::isnan, std::fpclassify (zero, subnormal, normal, infinite, NN, or implementation-defined category)

```
double sin = std::sin(3.14/2);
double cos = std::cos(0);
double tan = std::tan(3.14/4);
std::println("sin = {}, cos = {}, tan = {}", sin, cos, tan);
```

sin = 0.9999996829318346, cos = 1, tan = 0.9992039901050427

```
double sqrt = std::sqrt(16.0);
std::println("sqrt(16.0) = {}", sqrt);

double log = std::log(2.718281828459045);
std::println("log(e) = {}", log);
```

sqrt(16.0) = 4
log(e) = 1

Value ranges

Type	Size in bits	Format	Value range	
			Approximate	Exact
character	8	signed (one's complement)	-127 to 127	
		signed (two's complement)	-128 to 127	
		unsigned	0 to 255	
integral	16	signed (one's complement)	$\pm 3.27 \cdot 10^4$	-32767 to 32767
		signed (two's complement)		-32768 to 32767
		unsigned	0 to $6.55 \cdot 10^4$	0 to 65535
	32	signed (one's complement)	$\pm 2.14 \cdot 10^9$	-2,147,483,647 to 2,147,483,647
		signed (two's complement)		-2,147,483,648 to 2,147,483,647
		unsigned	0 to $4.29 \cdot 10^9$	0 to 4,294,967,295
	64	signed (one's complement)	$\pm 9.22 \cdot 10^{18}$	-9,223,372,036,854,775,807 to 9,223,372,036,854,775,807
		signed (two's complement)		-9,223,372,036,854,775,808 to 9,223,372,036,854,775,807
		unsigned	0 to $1.84 \cdot 10^{19}$	0 to 18,446,744,073,709,551,615
floating point	32	IEEE-754	$\pm 3.4 \cdot 10^{\pm 38}$ (~7 digits)	• min subnormal: $\pm 1.401,298,4 \cdot 10^{-47}$ • min normal: $\pm 1.175,494,3 \cdot 10^{-38}$ • max: $\pm 3.402,823,4 \cdot 10^{38}$
	64	IEEE-754	$\pm 1.7 \cdot 10^{\pm 308}$ (~15 digits)	• min subnormal: $\pm 4.940,656,458,412 \cdot 10^{-324}$ • min normal: $\pm 2.225,073,858,507,201,4 \cdot 10^{-308}$ • max: $\pm 1.797,693,134,862,315,7 \cdot 10^{308}$

void

- Indicates **no value**, which is **common** in **function return types**
- Cannot declare a variable of type void
- **Not type-safe**; avoid in modern C++ (prefer templates or smart pointers)
- **Special use**: *void* in parameter list (C style) instead of empty brackets (); discouraged in modern C++

```
// void variables are not allowed
void x;

// function returning void
void sub();

// pointer to unknown type
void* p;
```

const qualifier

- The const qualifier specifies that a variable's value **cannot be modified** after initialization (runtime immutability)
- Applies to:
  - **Objects** (variables)
  - **Pointers**
  - **Function parameters**
  - **Member functions**

```
// pi cannot be changed
const float pi = 3.14; // pi is a constant integer
const int y = 20; // y is a constant integer
```

Exercises

- Signed vs unsigned overflow: what happens if you subtract 1 to 0u and to 0?

Derived types

Data types

Enums

Data types

Enumerations in C++

- An **enumeration (enum)** is a **user-defined type** consisting of a **set of named integral constants**, used to **represent** a collection of **related values in a readable way**
- **An *enum*:**
  - Improves **code readability** and **maintainability**
  - **Avoids** using **magic numbers**
  - **Provides type safety** (especially with *enum class*; we'll see it later)
  - Is useful for representing **states**, **options**, or **categories**

Traditional (unscoped) Enums

- Scope is global within the enclosing namespace
- Values are implicitly converted to *int*
- Weaker type checking

```
enum Direction { North = 1, South, East, West };
int direction = North; // Allowed

enum Color { Red = 0, Green, Blue };

Color color = Red; // OK
color = 0; // might be OK (depends on compiler), but unsafe

switch (color) {
    case Red:
        std::println("Color is Red");
        break;
    default:
        std::println("Color not Red");
        break;
}
```

Static arrays

Data types

Static arrays

- An **array** is a collection of elements of the **same type**, stored in **contiguous memory locations**
- The **number** of **elements in the vector is called *the length*** (or size) of the vector
- Each element is identified by an **index**, starting from 0 up to the $\text{length} - 1$
- **Elements** can be **accessed via indexing**, i.e., the [] operator: vet[i] denotes the element of the index vector *i*

```
int numbers[5]; // uninitialized

int arr_a[5] = {1, 2, 3, 4, 5};
int arr_b[] = {10, 20, 30}; // size inferred (3)

// Missing elements → zero-initialized
int arr_c[5] = {1, 2}; // {1, 2, 0, 0, 0}

int val = arr_a[0]; // first element
arr_a[2] = 42;    // modify third element
```

Static arrays

- Static arrays have **fixed size (length)**, known at compile time
- No built-in bounds checking (accessing out of range = undefined behavior)
- Arrays decay to pointers when passed to functions

```
int numbers[5]; // uninitialized

int arr_a[5] = {1, 2, 3, 4, 5};
int arr_b[] = {10, 20, 30}; // size inferred (3)

// Missing elements → zero-initialized
int arr_c[5] = {1, 2}; // {1, 2, 0, 0, 0}

int val = arr_a[0]; // first element
arr_a[2] = 42;    // modify third element
```

Multidimensional arrays

- Arrays can have **multiple dimensions**
- They are stored in in **row-major order**

```
// 3 rows, 4 columns
int matrix[3][4];

matrix[1][2] = 7;
```

Structs

Structured data types

Structures (or records)

```
struct < label> {
        <definition-of-fields>
};
```

- A **structure is** a finite **collection of variables that are not** necessarily **homogeneous** (of the same type), **each identified by a name**
  - It is used to **aggregate elements** (even of different types) under a single name and represent complex elements with different properties
- **Each element** contained is called **a field (or member),** and is **defined like a variable (type and name)**
- Defining **a structure involves defining a new data type**
  - **Defining a structure does not allocate memory**; it only introduces a new data type
  - **Space is allocated when** you **declare a variable** of the structure **type**

Defining structures

- **Fields can** be **initialized**:
  - **By position** (in declaration order)
  - **By name** (designated initializers, since C++20)
- Unlike C, **fields can have default values** using brace-or-equal initializers
- Unlike C, there is no need to prefix “struct” before the struct name or use typedef to do so

```
struct Point { int x; int y; };
```

```
Point p1 = {10, 20};          // by position
```

```
Point p2 = {.x = 10, .y = 20}; // by name (C++20)
```

```
struct Device {
   // Default values
   Status status = Status::Ok;
   Direction direction = Direction::North;
   unsigned uptime{0};
};
```

Field naming rules

- Field **names must be unique within a structure**. Therefore:
  - Different structures can have fields with the same name
  - Field names can coincide with variable or function names
- **Fields can be of different types** (primitive or other structures).
- **Fields cannot be self-referential by value** (infinite recursion), **but can hold pointers to the same type**

```
int x;

struct a {
   char x;
   int y;
};

struct b {
   int w;
   float x;
};
```

```
struct s {
   int a;
   // ERROR: self-reference
  s next;
};
// allowed
struct Node { int data; Node* next; };
```

Accessing the fields of a struct

- Once a structure variable has been defined, **individual fields** are **accessed** using **dot notation**

structure.field

- **Each field is used as a normal variable** of the type corresponding to that of the field

```
struct point {
   int x, y;
};
point p1, p2;
p1.x = 10;
p1.y = -2;
p2.x = 5;
p2.y = 7;
```

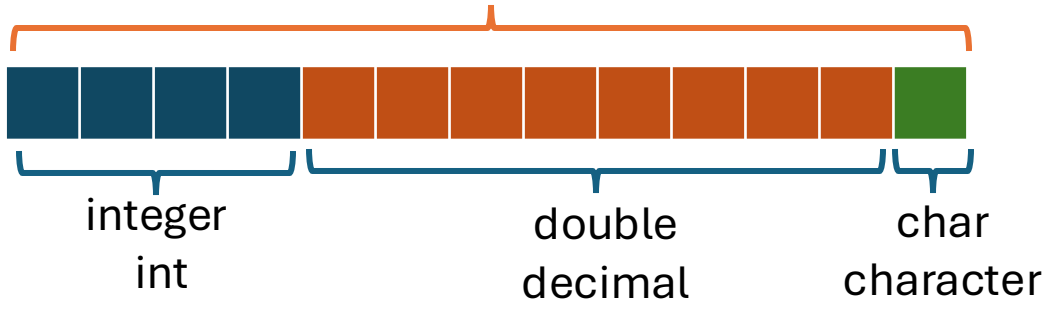
```
date {
   int day, month, year;
} d1, d2 ; /* contextual definition of variables d1 and d2
of type struct data */

d1.day = 25;
d1.month = 12;
d1.year = 2003;
printf("%d-%d-%d", d1.day, d1.month, d1.year);
```

Structure-Associated Memory

- **Each structure variable contains all fields**
- To coexist, **each field must have its own memory space**
- The size (*sizeof*) **of** a ***struct* variable is** (at least) **the sum of the size of all the fields it contains**
- Members are laid out sequentially with padding for alignment

```
struct Data {
   int integer;
   double decimal;
   char character;
};
```

Memory region associated with struct-type variables Data



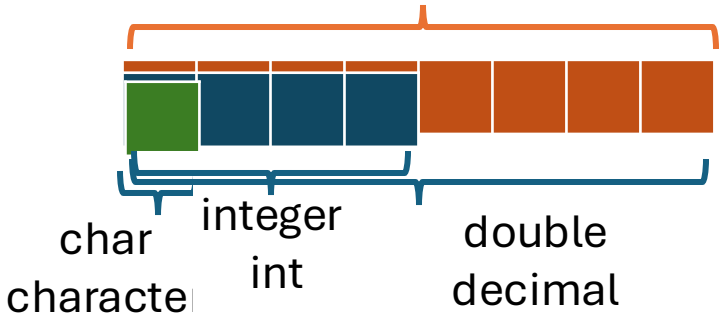
That is, the size is the sum of the dimensions of the elements in the *struct*

Union

- A **union** is a data structure that allows storing data of different types in the **same memory space**
- Unlike structures, where each member has its own memory area, **in a union all members share the same memory**
- This means only **one member is active at a time**
- The size of a union = size of its largest member

Memory region associated with variables of type union Data

```
union Data {
   int integer;
   double decimal;
   char character;
};
```

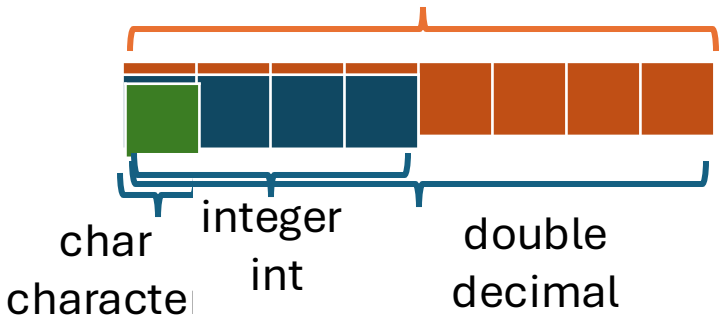


Union

- In C++, accessing a non-active member is **undefined behavior** (unlike C, where it's implementation-defined)
- **Unions are obsolete: use std::variant** or std::bit_cast **for safe type punning**
- The class template std::variant represents a type-safe union

Memory region associated with variables of type union Data

```
union Data {
   int integer;
   double decimal;
   char character;
};
```



Pointers

Data types

Pointers

- A **pointer** is a special variable that stores a **memory address**, typically the address of another variable
- This address allows direct access and manipulation of data in memory
- To declare a pointer, you must **specify the type of data to which the pointer will refer, followed by an asterisk**
  - The type is necessary to understand the type of data staked and be able to operate it correctly
- Remember: a variable is characterized by its identifier (name), value, type, and address

```
float* pf;
```

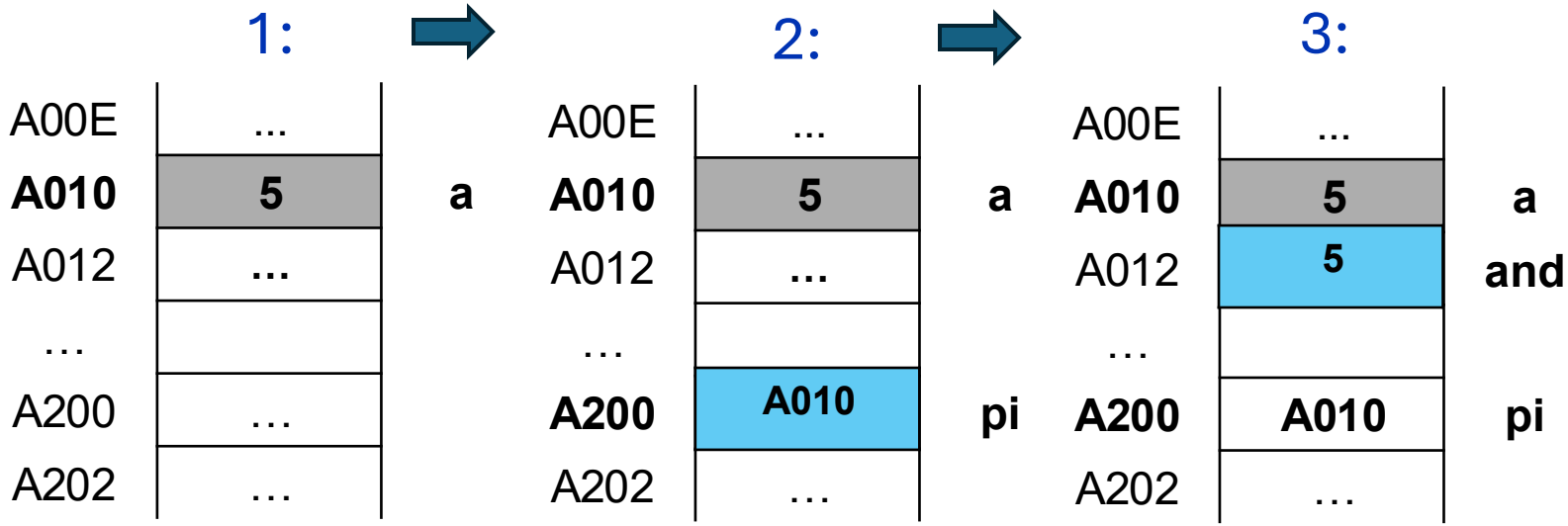
```
int* pi;
```

```
Device* p_dev;
```

Pointer Operations: Referencing and Read Access

- A pointer contains a memory address
- You can use **two operators**:
  - **& to get the address of a variable (of an l-value)**
  - *** to access the value referenced by the pointer**

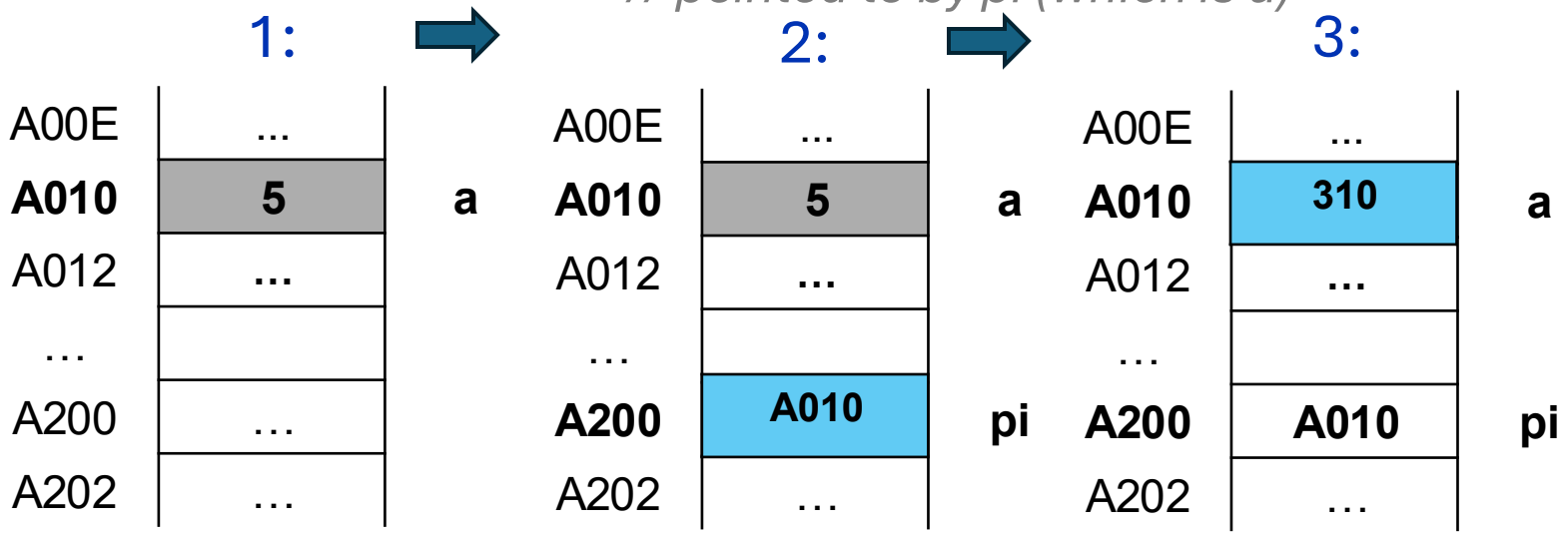
1. `int a = 5;`
2. `int *pi = &a;`
3. `int y = *pi; // read the value pointed to by pi`



Pointer Operations: Referencing and Write Access

- A pointer contains a memory address
- You can use **two operators**:
  - **& to get the address of a variable (of an l-value)**
  - *** to access the value referenced by the pointer**

1. `int a = 5;`
2. `int *pi = &a;`
3. `*pi = 310; // write inside the memory location // pointed to by pi (which is a)`



The * *symbol* in C

The **symbol *** (**asterisk**) in ANSI C can have **different meanings depending on the context:**

1. **Multiplication** operator between integers or reals
2. **Declaring** a **Pointer**
3. **Dereference operator** (read or write)

```c
#include <stdio.h>

int main() {
   int x = 5;
    Declaring a pointer to integer
   int *ptr = &x;  (and initialization with the address of x)

   // Multiplication between the value dotted by ptr and 10
   int result = 10 * *ptr;  *ptr is the value pointed to by ptr
                            // (i.e. the value of x)
                            // obtained by dereferencing
   printf("Result = %d\n", result);
    I write in x via deref of its pointer
   *ptr = 300;
}
```

Operators * and &

- The **indirection** (dereference) (*) **and address-of** (**&**) **operators allow** you **to interact with pointers**
  - * and & **are inverse functions**
  - They have **higher priority than binary operators**
- *** is right-associative (i.e., *****p* **is equivalent to** ***(*p)***
- **& can only be applied to l-values, and is not associative**
- If a is a variable *&a* is not a variable
  but if *pa* is a pointer **pa* is a variable!

Printing Pointers

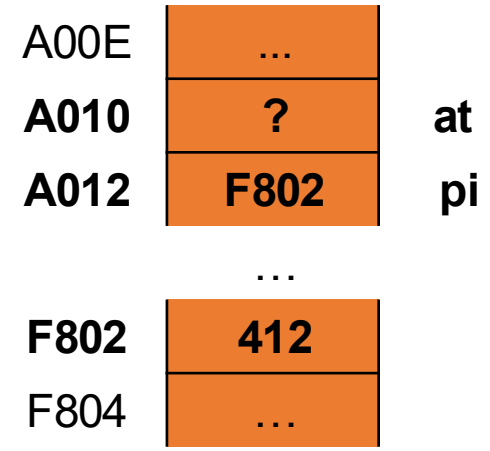
- **Pointer values** (memory addresses) **can be printed** using the *printf function* and the ***%p*** (hexadecimal) **format specifier**
- This is of little use in real applications anyway!

```
int a = 5;
int *pi = &a;
std::printf("value of a = %d\n", a);
// pi contains the address where
//    the value of a is stored.
printf("address of a = %p\n"
      "value of pi = %p\n", &a, pi);

// ... thus, *pi accesses the value of a
printf("value of *pi = %d\n", *pi);

// &*pi accesses the address of the value of a.
//    In fact, *pi accesses the value of a,
//       and &(*pi) accesses the address of a
printf("value of &*pi = %p\n", &*pi);

// *(&a) accesses the value of a. In fact:
//    &a accesses the address of a,
//       *(&a) accesses the value of a
printf("value of *&a = %d\n", *&a);
```

Initializing pointers

- **Pointers must be initialized** before they can be used
- It is a **mistake to dereference** an **uninitialized pointer** variable, which usually results in an access violation crash
  - That is, an attempt is made to access a “random” address that would not be accessible for security reasons or even does not exist

```
int a;
int *pi;

printf("%d\n", *pi);
*pi = a;
printf("%d\n", *pi);
```



Something totally random found in memory will be printed

void* pointers

- **C++ allows defining** a **pointer** to void (void*), which is **compatible with all pointer types**
- However, it **allows implicit** conversion **to** void* **from** T*, **but not** implicitly **from** void* **to** T* (**requires** an explicit **cast**)
- a void* **cannot be dereferenced directly,** you **must explicitly cast it** to the correct type before use

```
int  x{};

// OK (implicit to void*)
void* pv = &x;

// error in C++ (needs static_cast<int*>(pv))
int*  px = pv;
```

void* pointers

- Prefer *static_cast<T*>* over C-style casts for clarity and safety
- void* is mostly used for **low-level interop**, e.g., with C APIs or generic memory buffers
- In modern C++, use **templates**, **std::any**, or **std::variant** for type-safe polymorphism

```
int  x{};

// OK (implicit to void*)
void* pv = &x;

// error in C++ (needs static_cast<int*>(pv))
int*  px = pv;
```

Null pointers

- In C, ***NULL*** is **a macro for a null pointer constant**, **commonly defined as** ((void*)0) or 0 and implicitly converts to any object-pointer type
- **C++11 introduced *nullptr***, a **dedicated pointer literal** of **type std::nullptr_t**
  - It **converts to any pointer** (or pointer-to-member) **but not to integral types** (except bool)
- In C++, ***NULL*** remains an **implementation-defined null pointer constant,** typically 0 (or *nullptr* since C++11)
  - It **must not be defined as** ((void*)0) because C++ does **not** allow implicit conversion from void* to object pointer types

Prefer *nullptr* over *NULL* or *0*

- ***nullptr* unambiguously denotes "null pointer"**, not the integer zero, **and has type std::nullptr_t**
  - It **participates** in **overload resolution** distinctly from integers
  - **It is standard** C++11+ and will be portable going forward
- *NULL* is often an integer literal 0, which can select the wrong overload or be ambiguous
- Using it avoids legacy macro quirks

Exercise

- Define a struct
- Use pointers and references to operate on the struct

Accessing Elements of a Vector: Example

- In ANSI C a **vector** is **represented by the address of the first element**
- So the **vector variable** is **nothing more than a pointer**!
- For this reason, **you should** always **remember how many elements are in the vector**
- **We can act** on a vector **either through the [] operator or through pointer arithmetic**

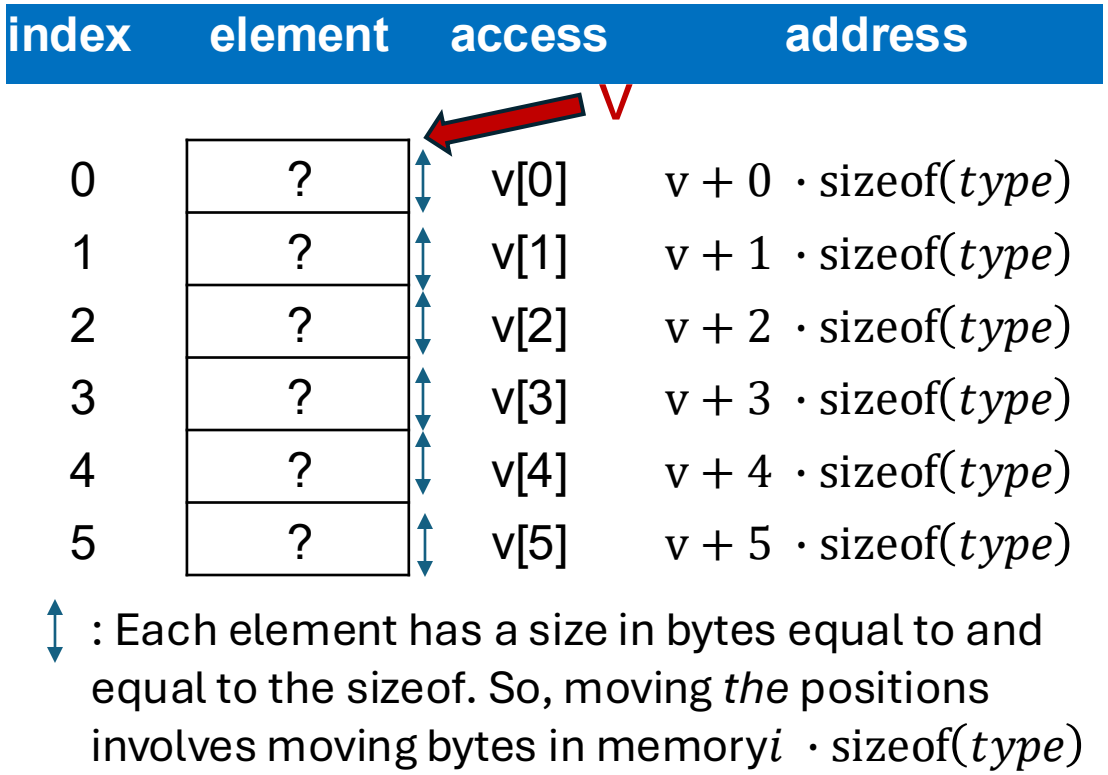
```
float *p = array;
printf("Pre-assignment:\n");
printf("\tArray value[0] %f\n", array[0]);
printf("\tValue of p = array %f\n", *p);

*p = 330.0f;

printf("Post-assignment:\n");
printf("\tArray value[0] %f\n", array[0]);
printf("\tValue of p = array %f\n", *p);
```

OUTPUT:
Pre-allocation:
Array Value[0] 1.000000
Value of p = array 1.000000
Post-assignment:
Array Value[0] 330.000000
Value of p = array 330.000000

Arrays

- An **array is** a **pointer** that indicates the **address of the first element**
- You can **access the others** with **an exactly calculable *offset**. In fact:
  - **The array** is **homogeneous** of a certain type **and each element occupies $\mathbf{sizeof(type)}$ *bytes***
  - To access the *i-th* **element of** v, just "move" by $i \cdot \text{sizeof(type)}$ bytes to the address $\text{v} + i \cdot \text{sizeof}(type)$
- This is what **the [] calculates operator does and accesses the address**:

$$\text{v}[i] = *(\text{v} + i \cdot \text{sizeof}(type))$$

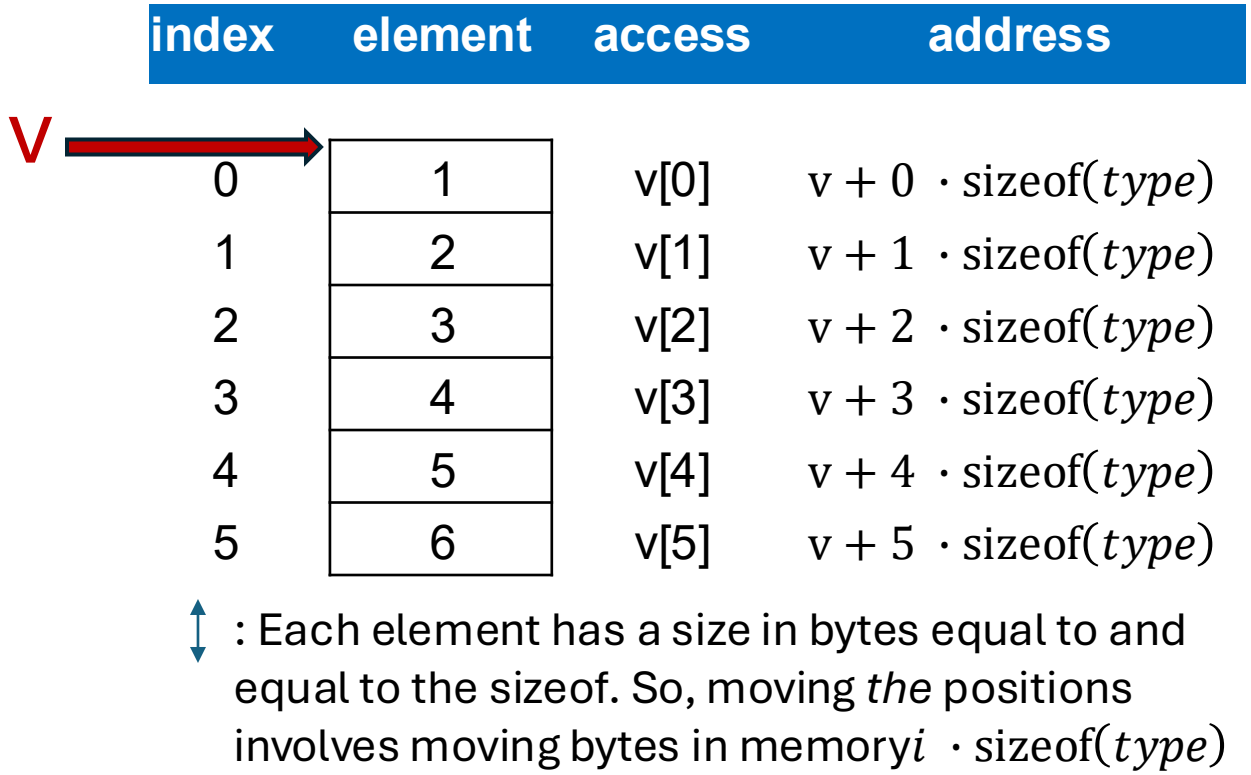


Accessing elements of an array: example

```
int v[] = {1, 2, 3, 4, 5, 6};
int *pi = v;
printf("sizeof(int) = %lu\n", sizeof(int));
printf("v = pi = %p\n", v);
printf("&v[0] = %p\n", &v[0]);
printf("&v[1] = %p\n", &v[1]);
```

OUTPUT:

sizeof(int) = 4
v = pi** = 0x16d82b2**00
*&v[**0**] = 0x16d82b2**00***
*&v[**1**] = 0x16d82b2**04***
*&v[**2**] = 0x16d82b2**08***
*&v[**3**] = 0x16d82b2**0c***
*&v[**4**] = 0x16d82b2**10***
*&v[**5**] = 0x16d82b2**14***



Pointer operators

- **Arrow operator ->** is syntactic sugar: e.g., (p->member) === (*p).member
- **Equality**: ==, != are well-defined for pointers of compatible types (incl. nullptr)
- **Relational**: <, <=, >, >= are defined **only** for pointers into the **same array** (or one-past)
- Comparing unrelated objects with relational operators is **unspecified behaviour**

Pointer Arithmetic

- In C, you **can perform pointer arithmetic operations** such as **increment** or **decrement to move** between **memory locations**
  - This is especially useful when working with arrays, strings, or structures
- This **pointer arithmetic** is a **powerful** and **flexible feature** of the language that allows efficient accesses
- However, **it is important to be careful to avoid errors** such as accessing unallocated memory or creating invalid pointers

Pointer operations

- **Dereference (*)**
- **Assignment** (pointers of same type)
- **Comparison** == and !=
  - They compare the addresses, **not** contained values
- **Arithmetic**, but a subset
  - **increase** (++) and **decrease** (--) pointed position
  - **Adding** (+, +=) and **subtracting** (-, -=) **an integer to a pointer**
  - **Subtraction** (-) **between pointers**

```
double someValue = 3.1415;
double *ptr = &someValue;
printf("Valore di someValue = %f\n", someValue);
*ptr = 2.71828;
printf("Valore di someValue = %f\n", someValue);

*ptr = (double) 2;
if (ptr == &someValue)
   printf("I due puntatori sono uguali\n");

double someOtherValue = 2;
printf("%f == %f\n", *ptr, someOtherValue);
if (ptr != &someOtherValue)
   printf("%p != %p\n", ptr, &someOtherValue);
```

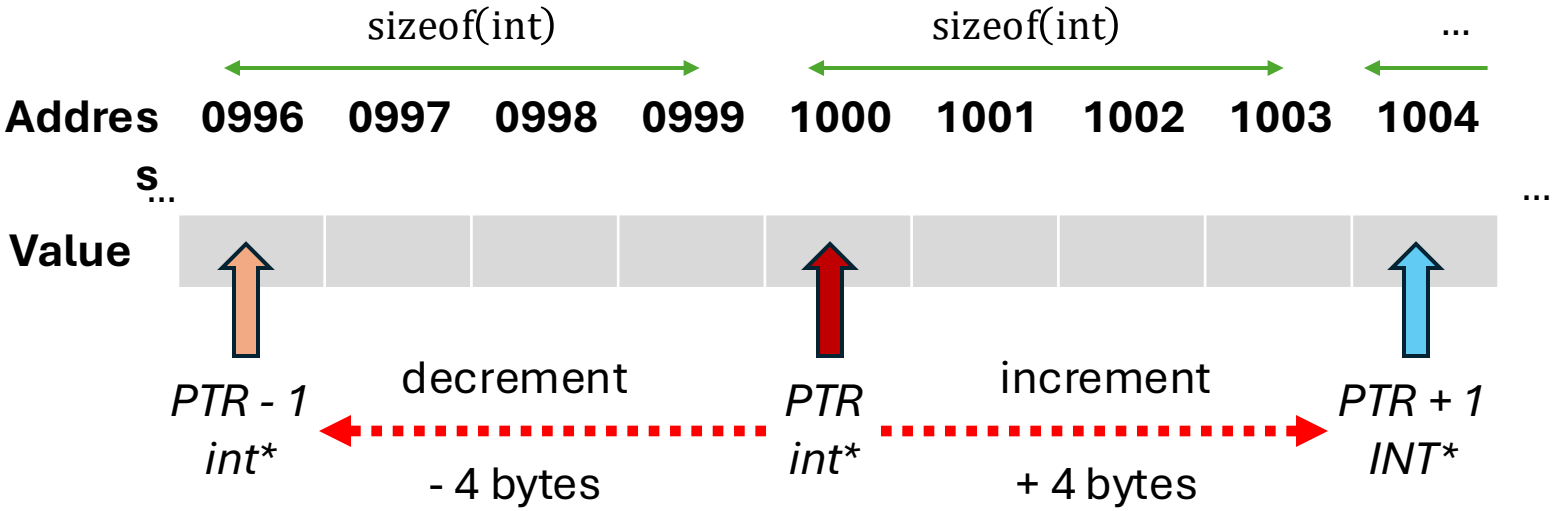
OUTPUT:
The two pointers are equal (ptr == &someValue)

2.000000 == 2.000000
0x16f1d7260 != 0x16f1d7250

Incrementing and decrementing pointers: adding and subtracting integers

- In **the addition** (subtraction) **between** a **pointer and** an **integer**, the **pointer** is **moved forward** (backward) **by a number of elements equal to the added integer, taking into account the size of the** data **type** the pointer points to
- In other words, the integer is **converted to** an **offset** sufficient to "skip" the entire memory space occupied by the bulleted type to the next element

Example pointer to integer: int* ptr



In this example, we are assuming that $\text{sizeof}(\text{int}) = 4$

More generally, therefore: $ptr + i$ skip elements, and$i$ is obtained by adding to the address of the pointer $i * \text{sizeof}(\text{int})$
If the pointer is not of type int* but of a generic type*, it is considered instead $i * \text{sizeof}(\text{type})$

Incrementing and decrementing pointers

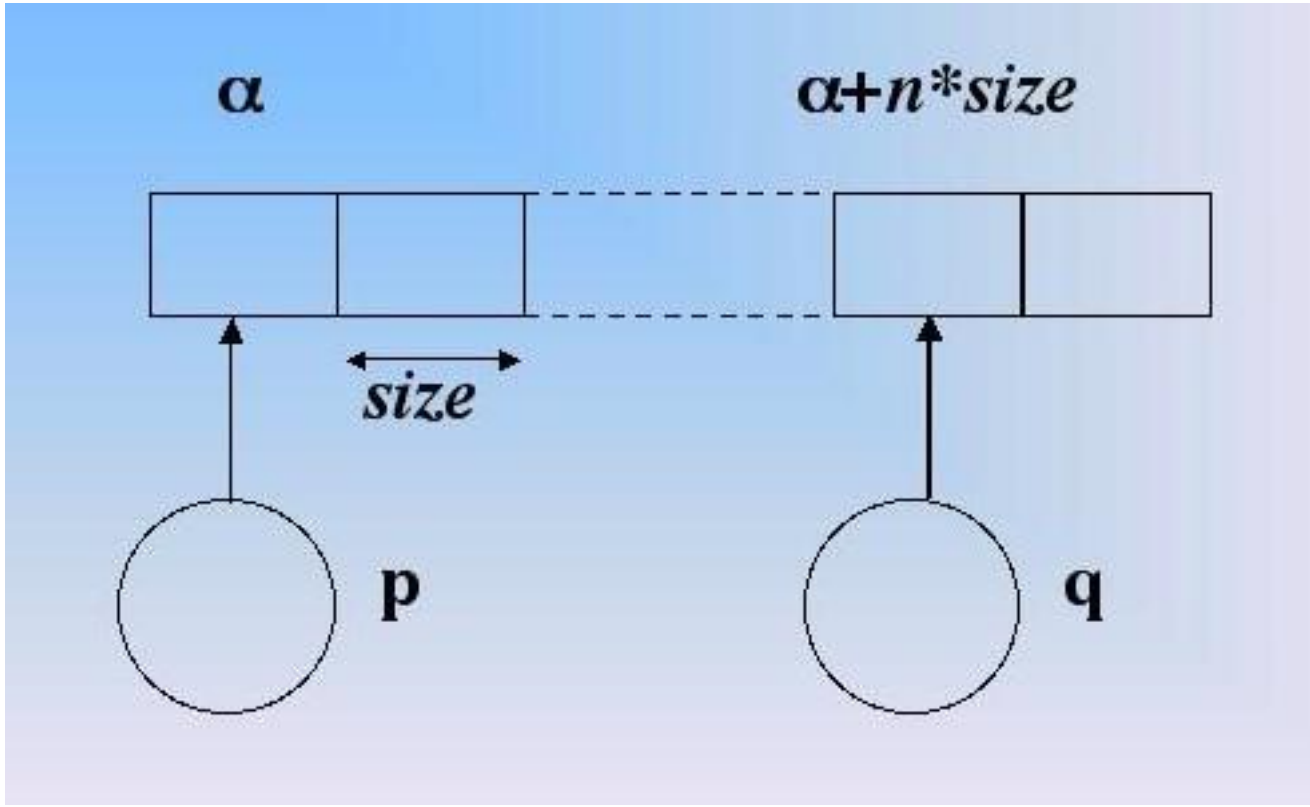
This type of operation is **useful**, specifically, **for operating on arrays**

In fact, arrays are **homogeneous** and **contiguous in memory**!

```
int numbers[] = {10, 20, 30, 40, 50};

int *ptr = numbers;

printf("Item #1: %d\n", *ptr);

// increment the pointer by 1
ptr++;  ptr = ptr + 1
printf("Item #2:%d\n", *ptr);

// increment the pointer by 2
ptr += 2;
printf("Item #4: %d\n", *ptr);

// decrease the pointer by 1
ptr--;  ptr = ptr - 1
printf("Item #3: %d\n", *ptr);

// It also decrements the pointer by 1
printf("Item #2: %d\n", *(--ptr));
```

Pointer Arithmetic

- In ANSI C, the sum of two pointers does not provide a meaningful semantic result
- Pointer arithmetic is used to move through memory locations in an array
- The sum of two pointers is not semantically significant because it does not represent a physical amount of memory to be moved
- The sum of a pointer and an integer is well defined



Exampleof a summing of an integer

1. Assigning the memory address of the first element of the vector to the *ptr variable*
2. I add two integer memory cells, then $2 \cdot \text{sizeof(int)}$
2. *newPtr* contains the address of position item 2

```
int array[5] = {1, 2, 3, 4, 5};
int *ptr = array;
int *newPtr = ptr + 2;

printf("&array[2] == &array[0] + 2: %s\n",
     &array[2] == newPtr ? "true" : "false");
printf("Value punctuation da newPtr: %d\n",
*newPtr);
```

OUTPUT

&array[2] == &array[0] + 2: true

Value Backed By newPtr: 3

Example of summing an integer

1. Assigning the memory address of the third element of the vector to the *ptr variable*
2. I add two integer memory cells, then $2 \cdot \text{sizeof(int)}$
2. *newPtr* contains the address of position element 5

```
int array[5] = {1, 2, 3, 4, 5};
int *ptr = &array[2];
int *newPtr = ptr + 2;

printf("&array[4] == &array[2] + 2: %s\n",
     &array[4] == newPtr ? "true" : "false");
printf("Value punctuation da newPtr: %d\n",
*newPtr);
```

OUTPUT

```
&array[4] == &array[2] + 2: true

Value backed by newPtr: 5
```

Pointer arithmetic: sum and difference between pointers

- Given *p* and *q* **two variables** of type **pointer to a certain type *T***
- **Direct summing between two pointers is not allowed and has no practical meaning**
- *The p – q* **subtraction** denotes an **integer** that represents **the number of cells between *p*** and ***q***
  - If ***q* precedes *p*** the denoted integer **is a negative number**
  - **It is calculated by subtracting the addresses and dividing by the sizeof(T)**

```
int array[5] = {1, 2, 3, 4, 5};
int *p = &array[2],
   *q = &array[5];
long distance = q - p;
printf("distance = q - p = \n");
printf("= (%p - %p) / %ld = %ld\n", q, p,
sizeof(int),
     distance);

printf("distance = p - q = \n");
printf("= (%p - %p) / %ld = %ld\n", p, q,
sizeof(int),
     p - q);
```

OUTPUT:

distance = q - p =
= (0x16d043204 - 0x16d0431f8) / 4 = 3

distance = p - q =
= (0x16d0431f8 - 0x16d043204) / 4 = -3

Accessing the fields of a struct

The dot operator " . ", like the *[]* operator of arrays, **calculates a** memory **offset**

Each field of a structure **is** in fact **stored in a specific memory location**, with an offset from the beginning of the structure

The offset is **determined by the size** and **order of the fields** in the outline

The address of the *struct variable* coincides with that of the first field

Arrow operator -> is syntactic sugar for (*p).member

```
struct Person {
   char name[50];   // 50*1 byte
   int age;         // 4 bytes
   float height;    // 4 bytes
};
```

Declaration of the structure

```
struct Person p1 = {
      .name = "John Doe",
      .age = 30,
      .height = 1.75f,
};
```

Declaring a Variable of the "Person" structure type

```
printf("Address structure variable: &p1 = %p\n", &p1);
printf("Field Address \"name\": &(p1.name) = %p\n", &p1.name);
printf("Address field \"age\": &(p1.eta) = %p\n", &p1.age);
printf("Address field \"height\": &(p1.height) = %p\n", &p1.height);
```

OUTPUT:
Variable Structure Address: &p1 = 0x16d60b**1cc**
Address field "name": &(p1.name) = 0x16d60b**1cc**
Address field "age": &(p1.eta) = 0x16d60b**200**
Address field "height": &(p1.height) = 0x16d60b**204**

const and pointers

Remember that in C, types should be read from right to left!

- *const* can apply to the **pointee**, the **pointer**, or both
- **Top-level const**: applies to the object itself (cannot reassign)
- **Low-level const**: applies to the object being pointed to

Declaration	Meaning
const int* p	Pointer to **const int** (value read-only)
int* const p	**Const pointer** to int (address fixed)
const int* const p	Const pointer to const int

```
int x = 0;
int* const p = &x;    // top-level const
const int* q = &x;    // low-level const
```

```
int a = 1, b = 2;
const int* p = &a; // *p cannot change

p = &b;          // ok
// *p = 3;        // error: cannot modify through p

int const* q = &a; // same as const int*

int* const z = &b;
// z = &a;        // error: cannot change z
```

Exercises

- Implement the following functions

```
// index-based
int sum(const int a[], int n);
int find_first(const int a[], int n, int value);   // -1 se non trovato
int min_index(const int a[], int n);
int max_index(const int a[], int n);
void reverse(int a[], int n);
// pointer-based
int sum(const int* first, const int* last);
const int* find_first(const int* first, const int* last, int value);
int* reverse(int* first, int* last); // return first per chaining
void add_offset(int* begin, int* end, int offset);
```

- Implement a sorting algorithm

References

Data types

References

- A **reference** is an **alias for an existing object**
- Unlike pointers, they **cannot be null**, must be **initialized at declaration and cannot be reseated** to refer to another object
- **Declared with & after the type**
- They are aliases: **no operators required** unlike pointers
- They be thought of as a **constant pointer** (not a pointer to a constant value!) **with automatic indirection**, i.e., the compiler will apply the * operator for you

```
int x = 10;
int& ref = x;   // ref refers to x

ref += 5;     // modifies x
std::cout << "x = " << x << '\n';
std::cout << "ref = " << ref << '\n';
```

```
x = 15
ref = 15
```

```
// constant reference:
const int &r1 = x; // r1 refers to x
// error: cannot modify through r1
// r1 += 5
```

Pointers vs References

Feature	Reference	Pointer
Syntax	int& r = x;	int* p = &x;
Null allowed?	No	Yes (*nullptr*)
Must init?	Yes	No
Reseatable?	No	Yes
Dereference	Implicit	Explicit (*p)
Arithmetic	Not allowed	Allowed (within array)

Pointers vs References: Best practices

- Prefer **references when**:
  - **Object must not be *null***
  - **No reseating** is needed
- Use **pointers when**:
  - **Nullability** is **meaningful**
  - **Dynamic allocation or reseating** is required

l-value vs r-value references

- **lvalue references** *(T&)* bind to lvalues but **cannot bind to rvalues**
- **const lvalue references** *(const T&)* can **bind to l-values and rvalues** (extending their lifetime), common for function parameters
- **rvalue references** *(&&)* bind to **rvalues** and were introduced with C++11
  - Enable **move semantics** and **perfect forwarding**

Remember:

- **Lvalue** has an identity (addressable), persists beyond the expression
- **Rvalue**: temporary, no persistent address, often immutable like string literals

```
// lvalue reference (&)
int& r = 5;     // ✗ error
```

```
// binds to temporary (an r-value)
const int &r2 = x + 5;
std::cout << "r2 = " << r2 << '\n';
// prints 20
const double& r3 = 3.14;
```

```
int&& rr = 5;   // ok
```

Lifetime of temporaries

Reminder: a temporary object is created by:

- Returning a value from a function
- Evaluating an expression that produces an rvalue (e.g., x + y, std::string("hi"))
- Casting or calling a constructor inline

- **A temporary lives until the end of the full expression** in which it was created
- **If** a temporary is **bound to a const lvalue reference, its lifetime is extended to match the reference**
- However, binding to subobjects (e.g., s[0] from a temporary s) does not extend lifetime

```
const string& ref = string("Hello");
// temporary lives until ref goes out of scope
```

```
char* p = nullptr;
{
    string&& str = string("Hello");
    println("String: {}", str);

    p = &str[3];
}
// p is dangling pointer
// Undefined behavior
std::println("Char at p: {}", *p);
```

lvalues and *rvalues* discussion

- **Rule:**
  - A **non-const lvalue reference (T&)** cannot bind to an rvalue (temporary)
  - A **const lvalue reference (const T&)** *can* bind to an rvalue
- The reason **rvalues (temporaries) can bind to const T&** is a deliberate **design choice** in C++ **to allow efficient and safe passing of temporaries** without unnecessary copies
- *Non-const T&* cannot bind to rvalues because **mutation of a dying object is nonsensical**

«I want to live forever in AI»

- **What people think will happen**:

```
bool uploadConsciousness(Consciousness& conscience) {
        /* ... */
}

uploadConsciousness(conscience);
```

- **What will actually happen**:

```
bool uploadConsciousness(Consciousness conscience) {
        /* ... */
}

uploadConsciousness(conscience);
```

Exercises

- Implement the following:
  - `void inc_by_value(int x);`
  - `void inc_by_ref(int& x);`
  - `void inc_by_ptr(int* x);`

- Define variables (e.g., primitive, structures)
- Use references to refer to such variables
- Define const refs

Type aliases

Type aliases

- **Type aliases** improve **readability** of complex types (e.g., function pointers, …)
- They **decouple interface** from **implementation details** (e.g., container or pointer flavor)
- In C++, both *typedef* and *using* can be used to create **type aliases**, but they differ in syntax, capabilities, and clarity
- Enable **generic aliases** with templates (modern C++)

typedef (C-style)

- ***typedef*** was introduced in C and available in C++ since the beginning
- Its syntax can be **not intuitive**, especially with complex types
- Use *using* in **modern C++** (C++11 and later)
- Reserve typedef only for legacy code or when working with C APIs

```
unsigned long * p;  // Pointer to unsigned long

// UL is now an alias for unsigned long
typedef unsigned long UL;
// var is of type unsigned long
UL var;

typedef UL* ULPointer;
typedef unsigned long * ULPointer;
ULPointer p1; // Same type as 'p'

// Matrix is a 9x9 array of doubles
typedef double Matrix[9][9];

typedef double (*FuncPtr)(double, double);
FuncPtr fn_ptr;

Matrix m;
m[2][1] = 3;
```

using keyword

- *using* is quite a **versatile keyword** in C++
- It can work as:
  - **Type alias declaration** (C++11)
  - A **declaration:** imports a name from a namespace
  - A **directive** imports *all* names from a namespace
  - A **directive for *enums*: to import enumerators** (C++20)

using (C++11 and later)

- *using* can declare type aliases in a way similar regular variable assignment
- Introduced in **C++11** as a more readable and powerful alternative
- Supports **template aliases**, which *typedef* cannot do
- Preferred in modern C++ for clarity and flexibility
- Syntax:

```
using typeAlias = type;
```

```
using UL = unsigned long;

using ULPointer = UL*;

using Matrix = double[9][9];

using FuncPtr = double (*)(double, double);

using BigInt = long long int;

using CharPtr = char*;
using CCharPtr = const char*;
```

More on the type system

auto (C++11)

- ***auto*** keyword **deduces** the **type** of a variable **from its initializer**
  - **Cannot be used without initialization**
- It reduces verbosity, especially with templates and long type names, and avoids mismatched types in templates and iterators
- Avoid when it hides important type semantics

```
auto x = 42;       // int

auto y = 3.14;     // double

auto s = "Hello";  // const char*
```

```
std::vector<int> v = {1, 2, 3};
for (auto it = v.begin(); it != v.end(); ++it) {
    std::cout << *it << '\n';
}
```

```
auto sum = [](int a, int b){ return a + b; };
std::cout << sum(3, 4.5); // works with int + double
```

Stronger type system than C

- C++ **introduces bool**, **references**, **function overloading**, and **templates**, all of which **require precise type matching**
- In C, **many conversions** (e.g., between void* and typed pointers) are implicit; **in C++**, they **require explicit casts**
- C++11 introduced ***nullptr* to eliminate ambiguity between integers and pointers**
  - Passing *0* or *NULL* to overloaded functions can cause ambiguity; *nullptr* enforces type safety

```
void* p = malloc(10);
int* ip;
ip = p; // OK in C but ERROR in C++

//In C++, you must write:
ip = static_cast<int*>(p);
```

```
void f(int*);
void f(int);
f(0); // OK
f(NULL); // AMBIGUITY
f(nullptr); // OK
```

Stronger type system than C: Function Overloading and Exact Matching

- C++ supports **function overloading**, so the compiler must resolve the correct overload based on exact types
- This makes type checking stricter because ambiguous or unsafe conversions lead to **compile-time errors** rather than silent conversions

```
void f(int*);
void f(int);
f(0); // OK
f(NULL); // ERROR: this matches two defs
f(nullptr); // OK
```

Stronger type system than C: more

- **Const-Correctness:**
  - C++ enforces **const correctness** more strictly than C
  - Example: Assigning a const int* to an int* is a compile-time error in C++, but may only warn in C

```
int i = 10;

int* p_int = nullptr;
const int* p_cint = nullptr;

p_int = &i;
p_cint = &i;

// error: assigning to 'int *' from 'const int *'
// discards qualifiers
p_int = p_cint;
```

- **Stricter type checking in templates**

Exercises

- Define some type aliases using typedef and using
- Use type inference (auto)

Variable assignation

Too many ways to cook an egg

Ways to initialize in C++: too many ways to cook an egg?

- C++ supports **multiple initialization syntaxes** for variables, especially for **fundamental types**
- All produce the same result (for fundamental types!!)
- These forms evolved for historical and technical reason

```
int a = 0;    // copy initialization
int b(0);     // direct initialization
int c{0};     // list initialization (C++11+)
int d{};      // value initialization (C++11+)
```

Ways to initialize in C++

- **0. Default initialization**: occurs when a **variable** is **declared without** an **initializer**
  - Behavior depends on storage duration:
    - **Automatic (local)**: indeterminate value
    - **Static/thread**: zero-initialized

```
int x;        // indeterminate (automatic)
static int y; // zero-initialized
```

```
int a = 0;    // copy initialization
int b(0);     // direct initialization
int c{0};     // list initialization (C++11+)
int d{};      // value initialization (C++11+)
```

Ways to initialize in C++

1. **Copy initialization** (=) looks like assignment but is actually initialization
2. **Direct initialization** (()) uses parentheses, often relevant for class types
3. **List-initialization** ({} , since C++11) was introduced in C++11 for **uniform initialization**
4. **Value initialization** ({} with no value) zero-initializes fundamental types

```
// Copy initialization
int a = 10;
std::string s1 = "Hello";

// Direct initialization
int b(10);
std::string s2("Hello");

// List initialization (since C++11)
int c{10};
std::string s3{"Hello"};
std::vector<int> v{1, 2, 3};

// Value initialization
int d{};      // a = 0
std::string s4{}; // empty string
```

Copy vs Direct Initialization

- int x = 0; → **copy-initialization syntax** but no actual copy for fundamental types
- int y(0); → **direct-initialization syntax**
- For **fundamental types**, both are equivalent
  - Both are **initializations**, not assignments;
  - There is no materialized temporary “0” that gets copied into v
- For **class types**, they can differ (e.g., explicit constructors)

```
// Copy-initialization
int x = 3.5;  // OK (x = 3)

int x2 = 10;

int y(20);

int z(y + 5);
```

List Initialization ({}) (C++11)

- **Braced initialization blocks narrowing conversions**
- **Prefer** {} **when** you want the compiler to **guard** you **against narrowing**
- Also useful for uniform initialization and aggregates

```
// Copy-initialization
int x = 3.5;  // OK (x = 3)

// Braced initialization does not
// allow narrowing
int y{3.5};   // error: narrowing conversion

int w{30};

int v{w + 5};

int v2{w + 5.5}; // error: narrowing conversion
```

https://en.cppreference.com/w/cpp/language/initialization.html

Examples

```
std::string s1;         // default-initialization

std::string s2();       // NOT an initialization!
                        // actually declares a function “s2”
                        // with no parameter and returns std::string

std::string s3 = "hello"; // copy-initialization
std::string s4("hello");  // direct-initialization
std::string s5{'a'};      // list-initialization (since C++11)
char a[3] = {'a', 'b'}; // aggregate initialization
                        // (part of list initialization since C++11)
char& c = a[0];         // reference initialization
```

Ways to initialize in C++

5. **Aggregate initialization** for arrays and aggregates (structs without private members or constructors) (it is now just list initialization since C++11)

6. **Designated Initializers (C++20)** for aggregates, specify members explicitly

```
// Aggregate initialization
int arr[3] = {1, 2, 3};

struct Point { int x, y; };
// Aggregate initialization
Point p = {10, 20};

// Designated initializer (since C++20)
Point p{ .x = 10, .y = 20 };
```

Practical guidance

- For **scalars** (e.g., int): pick **one** style and be consistent
- Remember that *T x = e*; is still **initialization**, not assignment: overloaded assignment operators are **not** involved

  int v = 0; is idiomatic and fine. No speed difference
- Use **{}** when you want **narrowing protection** or to **default-construct** clearly (*T t{};*)
- Prefer **()** when you must **select a specific constructor and beware its behavior**
  - E.g., vector<int>(10, 20) does not initialize a vector with two elements (10; 20) but a vector of 10 elements with value 20!

```
struct X {
   X(int) {}              // #1 (explicit or not changes copy-init)
   X(std::initializer_list<int>) {}  // #2
};

X a(0);    // picks #1 (direct-init)
X b = 0;   // picks #1 only if it's non-explicit (copy-init)        //
[3](http://naipc.uchicago.edu/2014/ref/cppreference/en/cpp/language/copy_initialization.html)
X c{0};    // prefers #2 (initializer_list) if viable               //
[4](https://en.cppreference.com/w/cpp/language/list_initialization.html)
```

Constant Initialization

- For **static storage duration** objects:
- Happens at compile time if the initializer is a constant expression
- Important for avoiding **static initialization order fiasco**

```
const int x = 42; // constant initialization
```